UVA | SCHOOL of DATA SCIENCE

Schema Enforcement in Streaming

Distributed Computing
School of Data Science
University of Virginia

Last updated: November 2, 2026

Reminder: What is a Schema?

A *schema* is a contract between producers and consumers of data.
It is a formal description of:

> Structure (fields, types, relationships)

> Constraints (required vs optional)

> Semantics (what the data means)

Examples of Schema

Database table definitions

API request/response formats

Event payloads

Serialized objects

Motivation

In streaming systems, data is:

Produced by many independent producers

Evolving continuously



Stream vs River | Creek vs Stream | Landscape Photography | Prints By Joseph C. Filer

Processed in real time, often without human intervention

Schema enforcement is even more critical in streaming systems

Motivation, contd.

Without schema enforcement:

A single malformed event can break downstream consumers

Silent data corruption propagates quickly

Analytics and models degrade before anyone notices

Schema Enforcement in Practice

Defining a formal structure for events

Validating events at ingestion or consumption

Controlling schema evolution over time

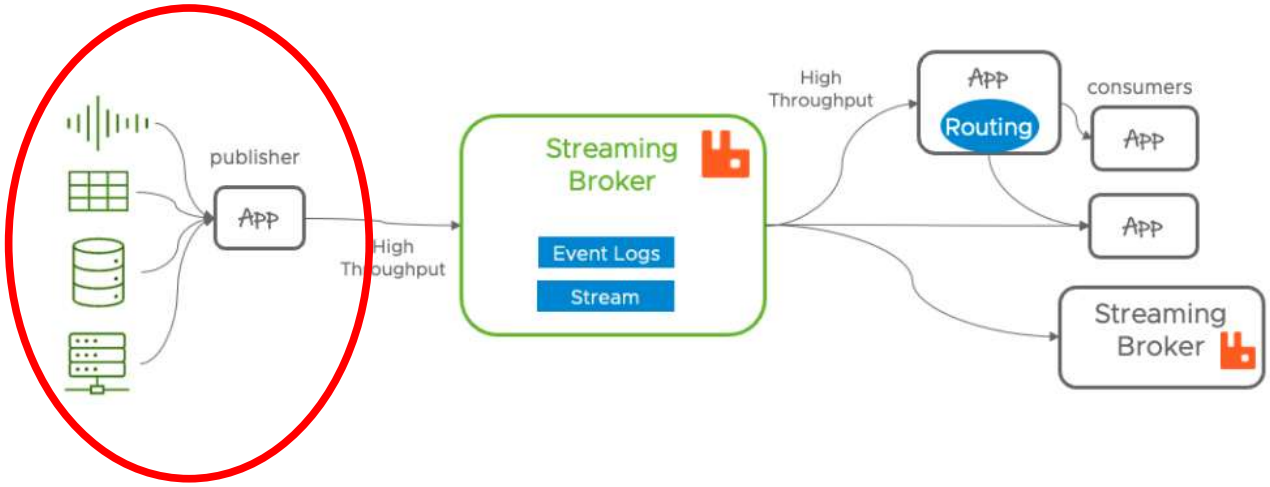
Streaming systems generally use schema-on-write
(validate when consuming)

Where Enforcement Happens

Producer-side enforcement

Producers serialize data using a predefined schema

Invalid events never enter the stream



Where Enforcement Happens: Producer

Producer-side enforcement

Pros

> Prevents bad data early

> Less downstream complexity

Cons

> Harder to evolve because:

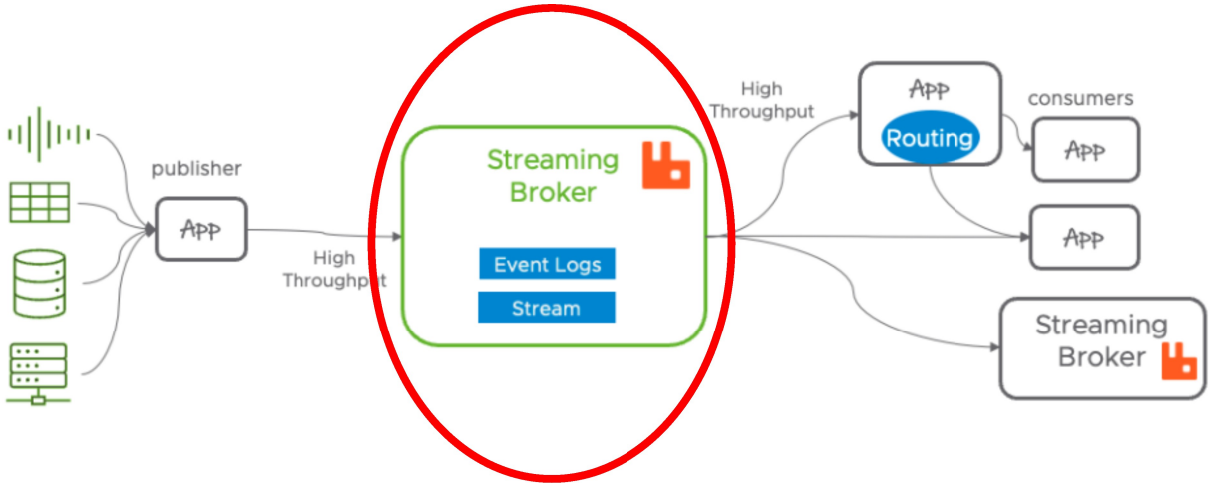
- Producers must strictly conform to schema before sending data
- Requires tight coordination across teams

Where Enforcement Happens: Broker

Broker / ingestion-layer enforcement

Stream broker or gateway validates events

Rejects or quarantines invalid messages



Where Enforcement Happens: Broker

Broker / ingestion-layer enforcement

Pros

> Centralized control

> Strong guarantees

Cons

> Adds latency

> Can become a bottleneck

Where Enforcement Happens: Consumer

Consumer-side enforcement

Consumers validate schema on read

Pros

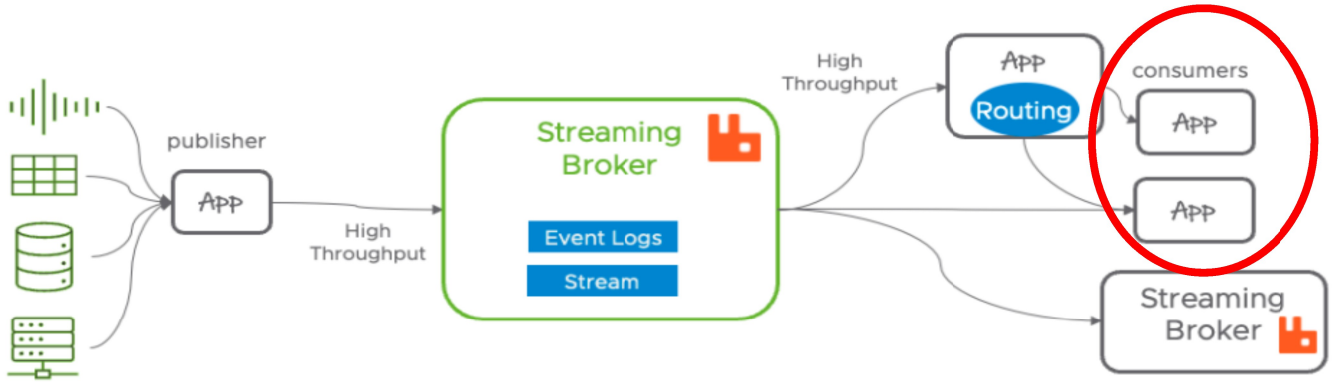
> Flexible

> Allows heterogeneous consumers

Cons

> Bad data already propagated

> Failure handling becomes complex



Schema Evolution in Streams

How to let schema change without breaking running pipelines?

Hard because:

> Old and new producers run simultaneously

> Consumers may lag behind

> Data cannot easily be “reprocessed”



Compatibility Models

Compatibility-based versioning is the dominant approach

Backward compatibility
New consumers can read old data

Safe changes include adding optional fields or providing defaults

When consumer sees version 1,
it provides default for missing email:

```
name  = Alice
age   = 30
email = null   (or a default value)
```

Version 1

JSON

```json
{
  "name": "Alice",
  "age": 30
}
```

Version 2

JSON

```json
{
  "name": "Alice",
  "age": 30,
  "email": "alice@example.com"
}
```

Compatibility Models, contd.

Forward compatibility

Old consumers can read new data

Requires new fields to be optional / ignorable

Full compatibility

> works in both directions

> harder to maintain but safest

Version 1

JSON

```
{
  "name": "Alice",
  "age": 30
}
```

Version 2

JSON

```
{
  "name": "Alice",
  "age": 30,
  "email": "alice@example.com"
}
```

Versioning Strategies

Strategy 1: Embed Schema Version

Embed schema version in each event

> Lightweight, self-describing messages

> Consumers handle multiple versions

> Event includes version

Version 1

```json
{
  "schema_version": 1,
  "event_type": "user_signup",
  "user_id": 123,
  "email": "user@example.com"
}
```

Version 2 (Additive Change)

```json
{
  "schema_version": 2,
  "event_type": "user_signup",
  "user_id": 123,
  "email": "user@example.com",
  "signup_source": "mobile_app"
}
```

Strategy 1: Embed Schema Version

Embed schema version in each event

> Lightweight, self-describing messages

> Consumers handle multiple versions

> Event includes version

Consumer Handling

Python

```python
def process_event(event):
    version = event["schema_version"]

    if version == 1:
        # Older schema
        user_id = event["user_id"]
        email = event["email"]
        signup_source = "unknown"  # default

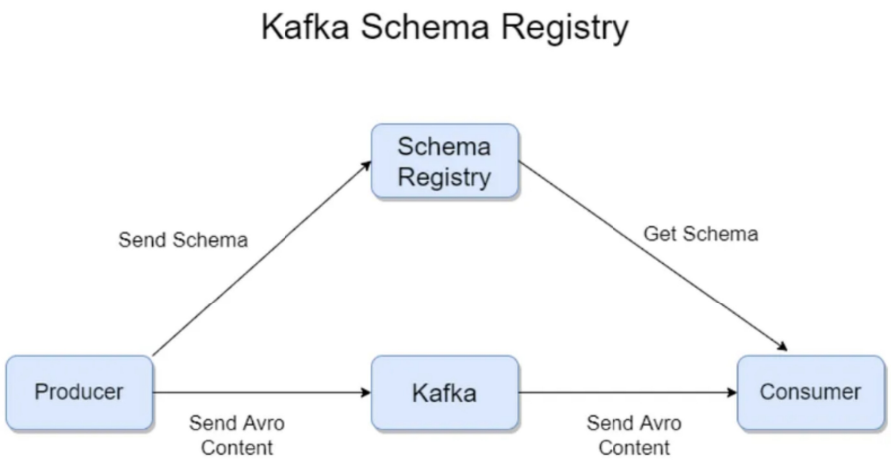
    elif version == 2:
        # New schema
        user_id = event["user_id"]
        email = event["email"]
        signup_source = event["signup_source"]

    print(user_id, email, signup_source)
```

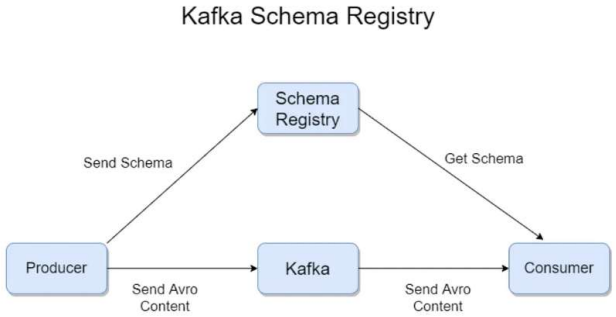
Strategy 2: External Schema Registry

Used by Apache Kafka + Apache Avro

> Centralized schema management and validation
> Enables compatibility checks (backward/forward)



Aside: Summary of Avro



Open source project managed by Apache

Provides data serialization and data exchange services for Apache Hadoop.

Self-describing data: stores data schema in file header using JSON

Serializes data into a small, fast binary format rather than text

Schema evolution: reconciles differences between schemas on the fly based on a strict set of rules

Strategy 2: External Schema Registry

How it works:

1 | Instead of including the full schema in every message,
which adds overhead, **producers send a compact schema ID**

2 | A centralized repository manages and validate schemas of messages

3 | Consumers use this ID to fetch corresponding schema from
registry to deserialize and process the data correctly

Failure Modes

Failure Mode: Silent Schema Drift

Producers change payloads without coordination

Consumers continue running but produce wrong results

Most dangerous failure: nothing crashes.

Failure Mode: Hard Failures in Consumers

Deserialization errors:

Failure to convert raw bytes or text back into an object or structured data according to expected schema or format.

Consumer crashes or restarts

Backpressure builds:

As consumer/network slows, queue builds and system can run out of memory or crash

Failure Mode: Partial Enforcement

Some producers enforce schemas, others don’t

“Mostly valid” streams are worse than invalid ones

Performance and Scale

> Validation adds latency

> Schema lookups add network overhead

> Binary formats trade readability for throughput

Key tradeoff:

Strong enforcement vs low-latency ingestion

How this Interacts with Analytics & ML

> Feature pipelines assume stable schemas

> Model training breaks silently with schema drift

e.g., all data is missing from a feature and it’s set to default value

> Real-time dashboards mislead decision-makers

Design Principles (Takeaways)

> Enforce schemas as early as possible

> Make schema evolution explicit and versioned

> Prefer backward-compatible changes

> Monitor for schema violations continuously

> Treat schemas as APIs, not documentation
(Schema is a contract between producer & consumer, like an interface.)